



# SICAE

Proyecto Final

Tecnologías para la Integración de Soluciones

Ramón Gómez Romero

15/06/2026

LISTI

Yael Lezama López y Karol Yunuen Cuervo Miranda

---



## Sistema de Control de Acceso de Estacionamiento SICAE

### Contenido

|                                                      |    |
|------------------------------------------------------|----|
| <b>Introducción</b>                                  | 3  |
| <b>Objetivos</b>                                     | 3  |
| <b>Explicación de microservicios</b>                 | 3  |
| Autenticación (AuthService)                          | 3  |
| Usuario (UserService)                                | 3  |
| Vehículo (VehicleService)                            | 3  |
| Estacionamiento (ParkingService)                     | 4  |
| <b>Arquitectura del Sistema</b>                      | 4  |
| Arquitectura de Microservicios                       | 4  |
| Arquitectura por Capas                               | 4  |
| <b>Diagramas</b>                                     | 4  |
| Diagrama General de Arquitectura                     | 4  |
| Diagrama de Despliegue                               | 5  |
| Diagrama de componentes de una arquitectura en capas | 6  |
| Diagrama de Flujo de Autenticación JWT               | 7  |
| <b>Diseño de API REST</b>                            | 8  |
| <b>Tecnologías</b>                                   | 9  |
| Tecnologías de Desarrollo                            | 9  |
| Tecnologías de almacenamiento                        | 9  |
| Tecnologías de Despliegue                            | 10 |
| <b>Pruebas</b>                                       | 10 |
| Pruebas de APIs                                      | 10 |
| Casos exitosos                                       | 10 |
| Casos de error                                       | 13 |
| Evidencias                                           | 15 |
| <b>Conclusiones</b>                                  | 19 |
| Resultados obtenidos                                 | 19 |
| Aprendizajes                                         | 19 |
| Problemas encontrados                                | 19 |



|                                                                                                                                                                           |    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figura 1: Arquitectura general del sistema SICAE y comunicacion entre microservicios y bases de datos.....                                                                | 5  |
| Figura 2. Despliegue en tres servidores logicos con contenedores Docker para aplicacion y base de datos.....                                                              | 6  |
| Figura 3. Capas internas usadas en el microservicio de Usuarios: controller, service, repository, model/dto y base de datos. ....                                         | 6  |
| Figura 4. Capas internas usadas en el microservicio de Vehículos: controller, service, repository, model/dto y base de datos. ....                                        | 7  |
| Figura 5. Capas internas usadas en el microservicio de Estacionamiento: controller, service, repository, model/dto y base de datos.....                                   | 7  |
| Figura 6. Flujo de autenticación: desde el envío de credenciales por parte del usuario hasta la generación y entrega del token JWT por parte del servidor.....            | 8  |
| Figura 7. Flujo de validación: desde la inclusión del token en los encabezados de las peticiones del cliente hasta la autorización de acceso en los microservicios.. .... | 8  |
| Figura 8. Login exitoso: el servicio devuelve datos del usuario y token JWT.....                                                                                          | 11 |
| Figura 9. Registro exitoso de vehiculo asociado al usuario.....                                                                                                           | 11 |
| Figura 10. Validacion correcta del vehiculo por placa e idUsuario. ....                                                                                                   | 12 |
| Figura 11. Registro de entrada con idMovimiento y espacio asignado. ....                                                                                                  | 12 |
| Figura 12. Registro de salida con calculo de costo total. ....                                                                                                            | 13 |
| Figura 13. Caso de error controlado: login con password incorrecto.....                                                                                                   | 14 |
| Figura 14. Error inicial de servidor durante pruebas de conexion y configuracion. ....                                                                                    | 14 |
| Figura 15. Error al consultar vehiculos antes de corregir la conexion del servicio. ....                                                                                  | 15 |
| Figura 16. Contenedores de usuarios, vehiculos y estacionamiento ejecutandose en Docker.....                                                                              | 15 |
| Figura 17. Login exitoso en Postman y generacion del token JWT. ....                                                                                                      | 16 |
| Figura 18. Consulta del perfil de usuario utilizando el encabezado Authorization. ....                                                                                    | 16 |
| Figura 19. Validacion correcta del vehiculo por placa e idUsuario. ....                                                                                                   | 17 |
| Figura 20. Consulta de espacios de estacionamiento disponibles.....                                                                                                       | 17 |
| Figura 21. Registro exitoso de entrada al estacionamiento.....                                                                                                            | 18 |
| Figura 22. Registro exitoso de salida y calculo del costo total.....                                                                                                      | 18 |



## Introducción

El presente documento describe el desarrollo del Sistema de Control de Acceso de Estacionamiento SICAЕ, implementado mediante una arquitectura de microservicios. El sistema permite autenticar usuarios, administrar vehículos registrados y controlar entradas y salidas de un estacionamiento.

La solución se organizó en tres proyectos Java desarrollados en NetBeans con Spring Boot: sicae-usuarios, sicae-vehiculos y sicae-estacionamiento. Cada microservicio trabaja con su propia base de datos, lo que permite separar responsabilidades y mantener una estructura más clara para consulta, mantenimiento y despliegue.

El flujo principal inicia con el inicio de sesión de un usuario. Después, con el token JWT generado, se pueden consumir los servicios de vehículos y estacionamiento. Para el despliegue se utilizó Docker Compose, separando cada servidor con su microservicio y su base de datos correspondiente.

## Objetivos

Objetivo general: desarrollar e implementar un sistema basado en microservicios para el control de acceso a estacionamiento, integrando autenticación, administración de vehículos, registro de movimientos y despliegue mediante contenedores.

Objetivos específicos: implementar un microservicio de usuarios para el inicio de sesión y generación de token JWT; implementar un microservicio de vehículos para registrar, consultar, actualizar y validar vehículos; implementar un microservicio de estacionamiento para registrar entradas y salidas; conectar cada microservicio con su base de datos; documentar la arquitectura, endpoints, pruebas y despliegue del sistema.

## Explicación de microservicios

### Autenticación (AuthService)

La autenticación se encuentra integrada dentro del microservicio sicae-usuarios. Este servicio recibe las credenciales enviadas desde Postman, busca el usuario en la base de datos PostgreSQL y valida la contraseña mediante BCrypt.

Cuando la contraseña es correcta, el servicio genera un token JWT. Este token contiene datos básicos del usuario, como idUsuario, usuario, rol y claveUsuario. Después se envía en el encabezado Authorization con el formato Bearer TOKEN para consumir los demás servicios protegidos.

### Usuario (UserService)

El servicio de usuarios administra la información relacionada con usuarios, roles, tipos de usuario y programa educativo. Su base de datos es PostgreSQL y se inicializa con el script sicaeUsuario.sql.

En el proyecto entregado, autenticación y usuarios están agrupados en el mismo microservicio para mantener la estructura de tres proyectos principales: usuarios, vehículos y estacionamiento. Esta decisión permite conservar el flujo solicitado sin crear más proyectos de los necesarios.

### Vehículo (VehicleService)

El microservicio sicae-vehiculos se encarga de las operaciones relacionadas con los vehículos registrados. Permite consultar vehículos por usuario, registrar nuevos vehículos, actualizar información, cambiar estatus y validar si una placa pertenece a un usuario.



Este servicio utiliza MySQL y la base se inicializa con sicaevehiculo.sql. Se agregó el campo estatus en la tabla vehiculo para poder activar o desactivar registros sin eliminarlos físicamente, lo cual es útil para conservar historial.

### Estacionamiento (ParkingService)

El microservicio sicae-estacionamiento controla los espacios y movimientos del estacionamiento. Al registrar una entrada, valida que el usuario exista, que el vehículo sea válido y que el espacio esté disponible. Si las validaciones son correctas, crea un movimiento y marca el espacio como ocupado.

Al registrar una salida, el servicio localiza el movimiento abierto, calcula el tiempo y el costo total con base en la tarifa por hora, actualiza el movimiento y libera el espacio. Este servicio se comunica con usuarios y vehículos por medio de peticiones HTTP.

## Arquitectura del Sistema

### Arquitectura de Microservicios

La arquitectura se dividió en tres microservicios independientes. Cada uno tiene una responsabilidad principal, su propio proyecto Java, su propio puerto y su propia base de datos. Esta separación permite que un cambio en vehículos no afecte directamente la base de usuarios o la lógica de estacionamiento.

Los puertos utilizados son 8081 para usuarios, 8082 para vehículos y 8083 para estacionamiento. En Docker, cada microservicio se ejecuta en un contenedor independiente y se comunica con su base de datos mediante el nombre del servicio definido en Docker Compose.

### Arquitectura por Capas

Cada microservicio sigue una estructura por capas. La capa controller recibe las peticiones HTTP y define los endpoints. La capa service contiene la lógica de negocio, validaciones y comunicación con otros servicios cuando es necesario. La capa repository contiene las consultas SQL mediante MyBatis. La capa model representa las entidades utilizadas por el sistema y la capa dto representa los datos de entrada o salida.

Esta organización facilita explicar el flujo interno: Postman llama a un endpoint, el controller recibe la solicitud, el service procesa la regla de negocio, el repository consulta o modifica la base de datos y finalmente el controller devuelve una respuesta JSON.

## Diagramas

### Diagrama General de Arquitectura

Este diagrama representa los componentes principales del sistema SICAЕ y la comunicación entre Postman, los microservicios y las bases de datos.

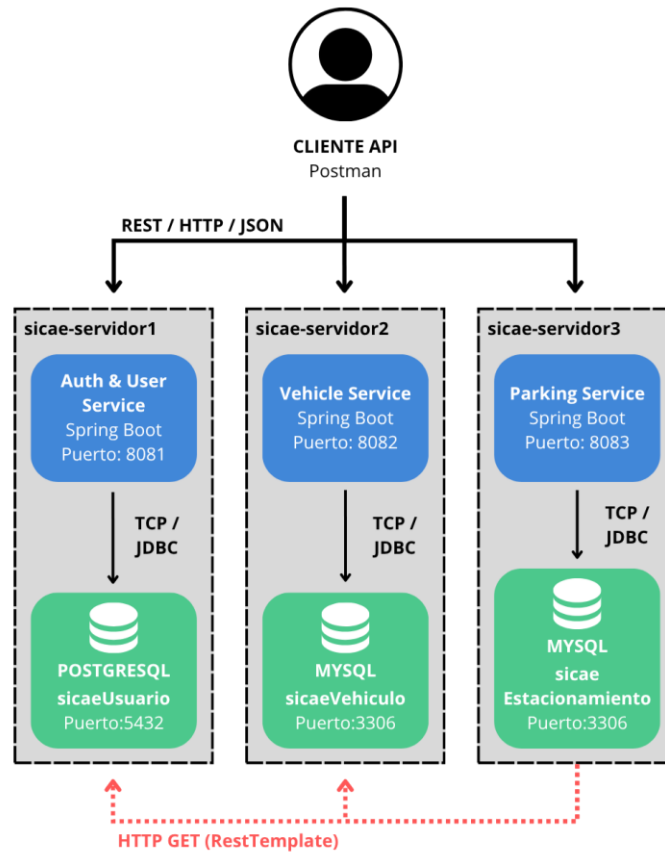


Figura 1: Arquitectura general del sistema SICAE y comunicacion entre microservicios y bases de datos

### Diagrama de Despliegue

El despliegue se organizó en tres servidores lógicos. El servidor 1 contiene sicae-usuarios y PostgreSQL; el servidor 2 contiene sicae-vehiculos y MySQL; el servidor 3 contiene sicae-estacionamiento y MySQL. En Docker se representan como contenedores separados para aplicación y base de datos.

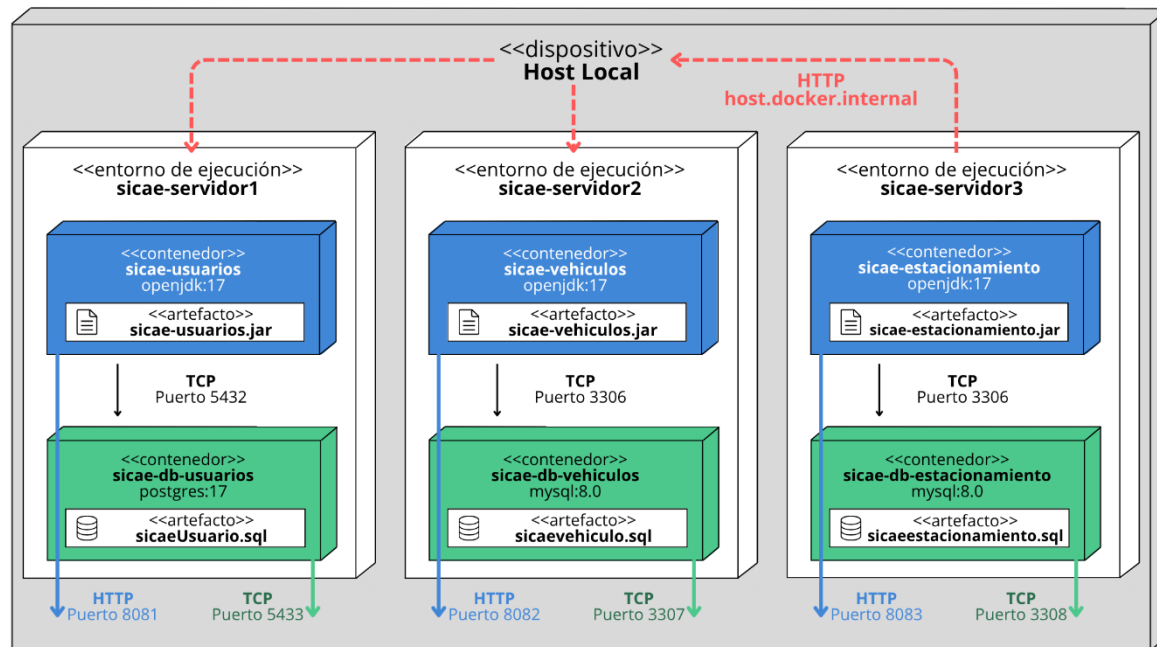


Figura 2. Despliegue en tres servidores lógicos con contenedores Docker para aplicación y base de datos.

### Diagrama de componentes de una arquitectura en capas

El diagrama de capas muestra la separación entre controller, service, repository, model/dto y base de datos. Esta estructura se repite en los tres microservicios, adaptándose a la responsabilidad de cada uno.

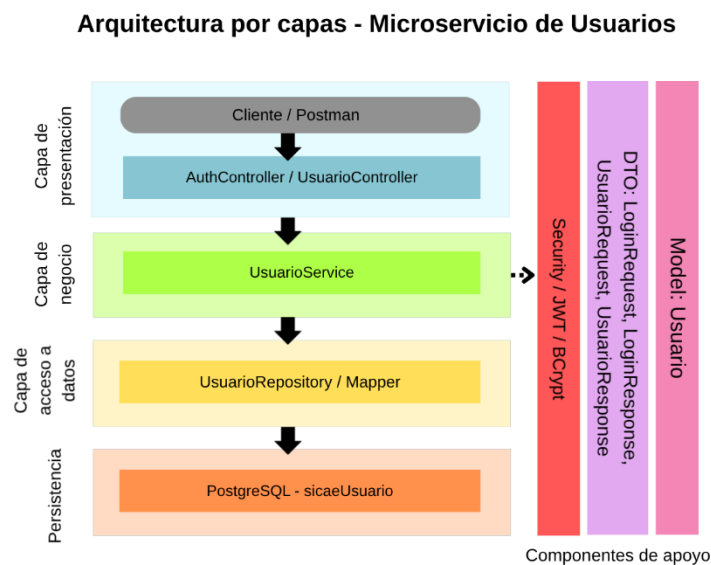


Figura 3. Capas internas usadas en el microservicio de Usuarios: controller, service, repository, model/dto y base de datos.

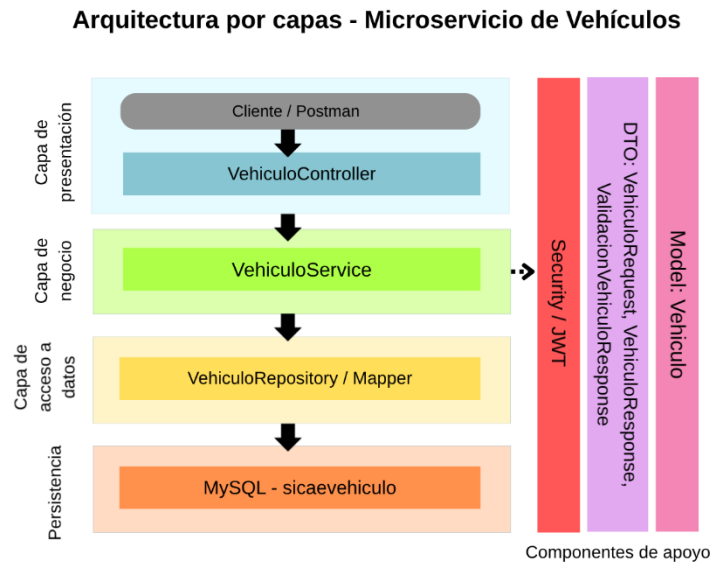


Figura 4. Capas internas usadas en el microservicio de Vehículos: controller, service, repository, model/dto y base de datos.

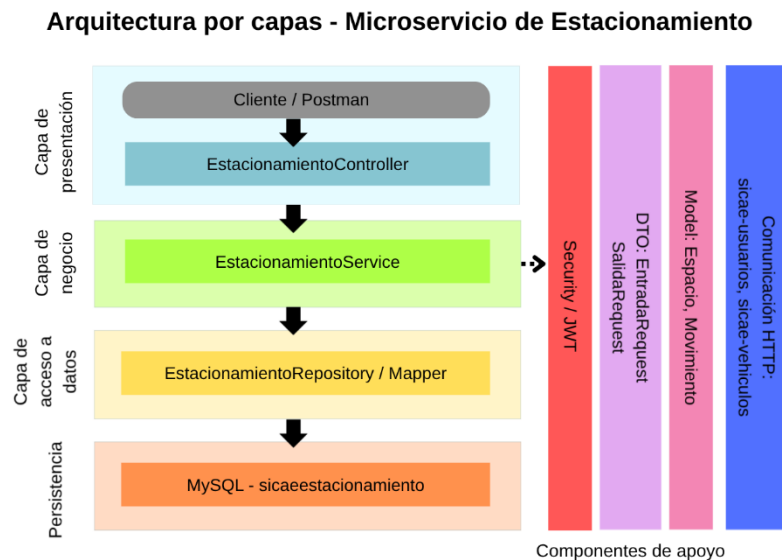


Figura 5. Capas internas usadas en el microservicio de Estacionamiento: controller, service, repository, model/dto y base de datos.

### Diagrama de Flujo de Autenticación JWT

El flujo de autenticación inicia cuando el usuario envía usuario y contraseña al endpoint de login. El microservicio de usuarios consulta PostgreSQL, valida la contraseña con BCrypt y genera un JWT. Después, el cliente envía ese token en el encabezado Authorization para consumir los endpoints protegidos.



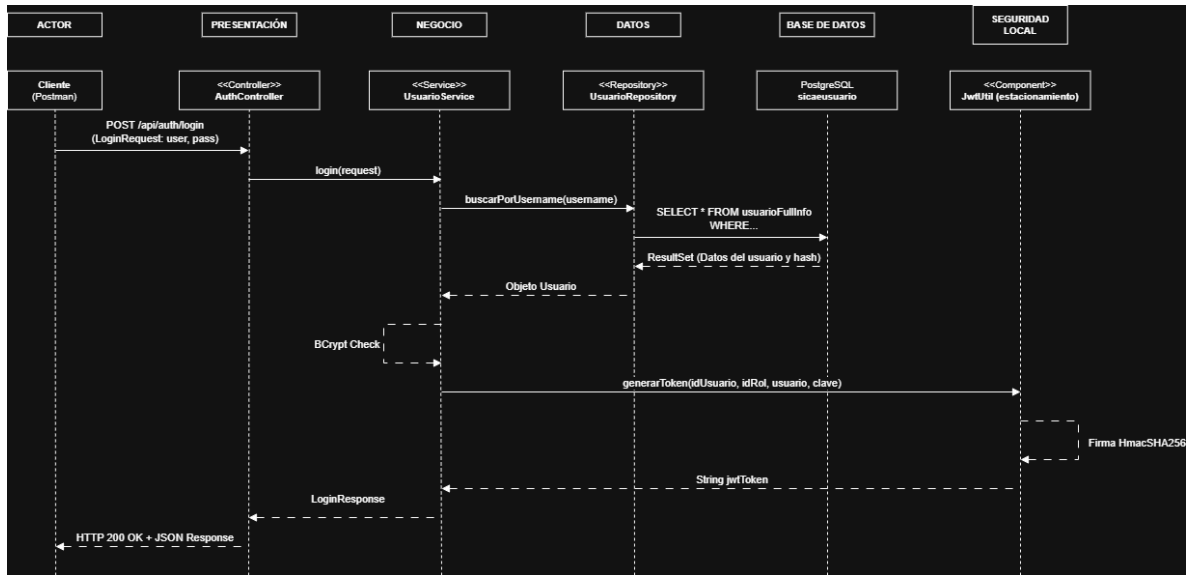


Figura 6. Flujo de autenticación: desde el envío de credenciales por parte del usuario hasta la generación y entrega del token JWT por parte del servidor.

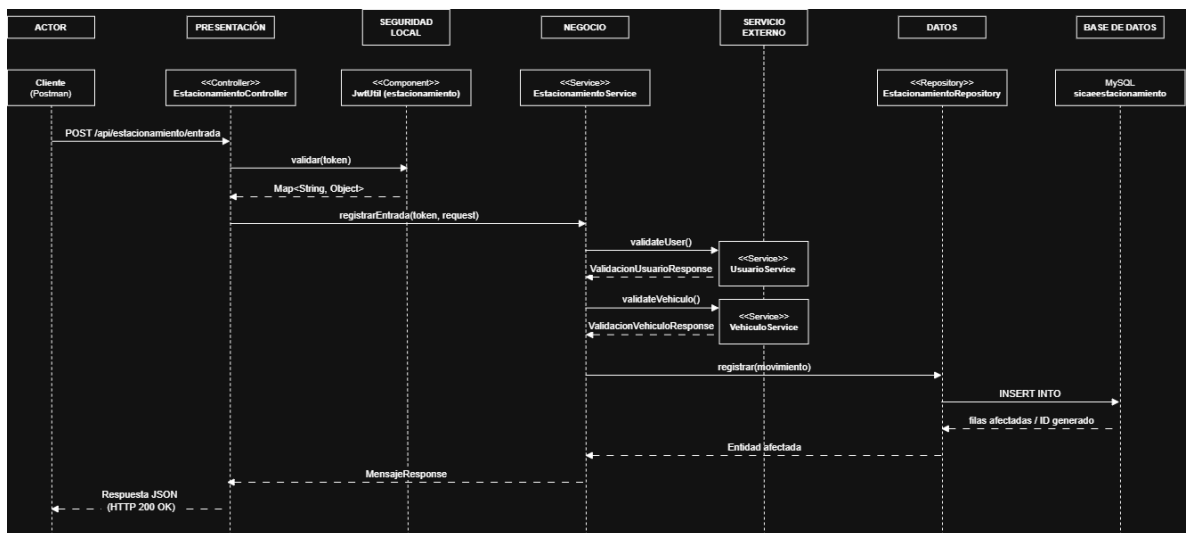


Figura 7. Flujo de validación: desde la inclusión del token en los encabezados de las peticiones del cliente hasta la autorización de acceso en los microservicios..

## Diseño de API REST

La API REST se diseñó usando métodos HTTP según la operación: GET para consultas, POST para registros, PUT para actualizaciones completas y PATCH para cambios parciales de estatus. Los datos se envían y reciben en formato JSON.

Los endpoints que requieren autenticación utilizan el encabezado Authorization con el formato Bearer PEGAR\_TOKEN\_AQUI. El token se obtiene desde el endpoint de login.

| Método | Endpoint        | Descripción                                 | Auth | Request                                   | Response                                                                           |
|--------|-----------------|---------------------------------------------|------|-------------------------------------------|------------------------------------------------------------------------------------|
| POST   | /api/auth/login | Inicia sesión. Valida usuario y password en | No   | {"usuario": "admin", "password": "admin"} | 200 OK: login correcto, datos del usuario y token JWT. Error: password incorrecto. |



*Universidad Veracruzana*  
*Facultad de Estadística e Informática*  
*Licenciatura en Ingeniería en Sistemas y Tecnologías de la Información*  
*Experiencia Educativa Tecnologías para la Integración de Soluciones*  
*Documentación Proyecto Final*  
*Periodo Febrero – Julio 2026*

|       |                                      |                                                                                   |            |                                                                                                              |                                                                                    |
|-------|--------------------------------------|-----------------------------------------------------------------------------------|------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
|       |                                      | PostgreSQL y genera el token JWT para las demás peticiones.                       |            |                                                                                                              |                                                                                    |
| POST  | /api/usuarios                        | Registra un usuario nuevo con rol, tipo de usuario y programa educativo.          | Bearer JWT | Datos del usuario en JSON.                                                                                   | 200 OK: usuario registrado. Error: datos incompletos o clave repetida.             |
| GET   | /api/usuarios/{idUsuario}            | Consulta el perfil de un usuario por identificador.                               | Bearer JWT | idUsuario en la ruta.                                                                                        | 200 OK: nombreCompleto, claveUsuario, email, rol, tipoUsuario y programaEducativo. |
| PUT   | /api/usuarios/{idUsuario}            | Actualiza la información general de un usuario existente.                         | Bearer JWT | JSON con los datos actualizados.                                                                             | 200 OK: usuario actualizado. Error: usuario no encontrado.                         |
| PATCH | /api/usuarios/{idUsuario}/estatus    | Cambia el estatus del usuario sin eliminarlo físicamente.                         | Bearer JWT | idUsuario en ruta y estatus a modificar.                                                                     | 200 OK: estatus actualizado.                                                       |
| GET   | /api/usuarios/validar/{claveUsuario} | Valida si existe un usuario activo con la clave indicada.                         | Bearer JWT | claveUsuario en la ruta.                                                                                     | 200 OK: valido true/false y datos básicos del usuario.                             |
| GET   | /api/vehiculos/usuario/{idUsuario}   | Lista los vehículos registrados para un usuario.                                  | Bearer JWT | idUsuario en la ruta.                                                                                        | 200 OK: arreglo de vehículos. Si no tiene vehículos, devuelve lista vacía.         |
| POST  | /api/vehiculos                       | Registra un vehículo con modelo, placa, color, año y descripción.                 | Bearer JWT | {"idUsuario":1,"idModelo":1,"placa":"ABC1234","color":"Azul","anio":2020,"descripcion":"Vehículo de prueba"} | 200 OK: vehículo registrado correctamente.                                         |
| PUT   | /api/vehiculos/{idVehiculo}          | Actualiza los datos de un vehículo registrado.                                    | Bearer JWT | JSON con idModelo, placa, color, año y descripción.                                                          | 200 OK: vehículo actualizado. Error: vehículo no encontrado.                       |
| PATCH | /api/vehiculos/{idVehiculo}/estatus  | Activa o desactiva un vehículo sin borrarlo de la base.                           | Bearer JWT | idVehiculo en ruta y estatus a modificar.                                                                    | 200 OK: cambio de estatus aplicado.                                                |
| GET   | /api/vehiculos/validar               | Valida que una placa exista y pertenezca al usuario indicado.                     | Bearer JWT | Query params: placa=ABC1234&idUsuario=1                                                                      | 200 OK: valido true, idVehiculo y mensaje Vehículo valido.                         |
| GET   | /api/estacionamiento/espacios        | Consulta espacios disponibles y ocupados.                                         | Bearer JWT | Sin cuerpo.                                                                                                  | 200 OK: lista de espacios con claveEspacio, tipo, ocupado y estatus.               |
| POST  | /api/estacionamiento/entrada         | Registra entrada. Valida usuario, vehículo y espacio antes de guardar movimiento. | Bearer JWT | {"claveUsuario":"fei","placa":"ABC1234","idEspacio":1,"tarifaHora":15.00}                                    | 200 OK: movimiento creado con idMovimiento, tiempoEntrada y claveEspacio.          |
| PUT   | /api/estacionamiento/salida          | Registra salida, calcula horas cobradas y costo total, y libera el espacio.       | Bearer JWT | {"claveUsuario":"fei","placa":"ABC1234"}                                                                     | 200 OK: movimiento actualizado con tiempoSalida y costoTotal.                      |

## Tecnologías

### Tecnologías de Desarrollo

Java se utilizó como lenguaje principal porque fue la tecnología trabajada durante las actividades del curso. Los proyectos se desarrollaron en NetBeans usando Spring Boot para exponer servicios REST.

MyBatis se utilizó para trabajar con consultas SQL de forma directa mediante repositorios Mapper. Esto permite mantener la consulta a base de datos visible y controlada.

JWT se utilizó para manejar autenticación entre peticiones. BCrypt se utilizó para validar contraseñas sin guardar la contraseña en texto plano.

### Tecnologías de almacenamiento

PostgreSQL se utilizó para la base de usuarios, ya que el script original de usuarios estaba preparado para este motor. MySQL se utilizó para las bases de vehículos y estacionamiento, respetando los scripts proporcionados.



Cada base de datos conserva su responsabilidad: sicaeUsuario almacena usuarios y roles; sicaevehiculo almacena marcas, modelos y vehículos; sicaeestacionamiento almacena espacios y movimientos.

### Tecnologías de Despliegue

Docker se utilizó para ejecutar los microservicios y bases de datos en contenedores. Docker Compose se utilizó para definir los servicios, puertos, variables de entorno, volúmenes y redes necesarios para iniciar cada servidor.

Se prepararon tres archivos docker-compose: uno para usuarios, uno para vehículos y uno para estacionamiento. Cada archivo representa un servidor lógico con su microservicio y su base de datos.

## Pruebas

### Pruebas de APIs

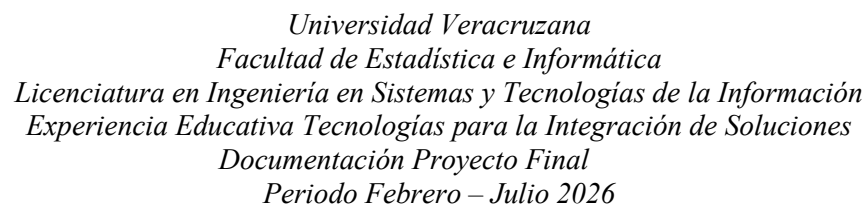
Las pruebas se realizaron con Postman consumiendo los endpoints de cada microservicio. Primero se ejecutó el login para obtener el token JWT. Después se utilizó el token en los endpoints protegidos de vehículos y estacionamiento.

También se realizaron pruebas desde el despliegue en Docker para confirmar que los contenedores, rutas, scripts SQL y comunicación entre servicios funcionaran correctamente.

### Casos exitosos

Las pruebas exitosas se ejecutaron siguiendo una secuencia completa para comprobar que los microservicios trabajan de forma coordinada. Primero se realizó el login para obtener el token JWT; después se consultó información del usuario, se registró un vehículo, se validó la placa, se consultaron espacios y finalmente se registró la entrada y salida del estacionamiento.

El resultado esperado en estos casos fue recibir respuestas 200 OK y objetos JSON con información coherente. En vehículos se confirmó el registro y la validación de la placa ABC1234. En estacionamiento se confirmó la creación del movimiento de entrada y el cálculo del costo total al registrar la salida.



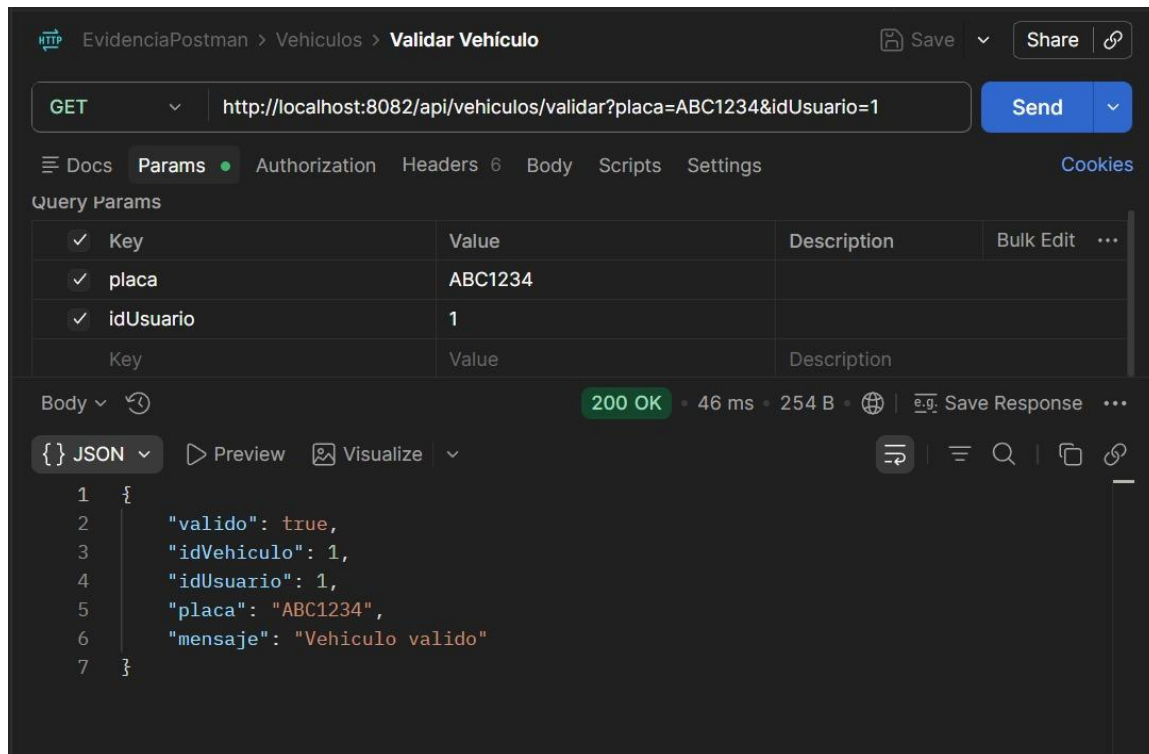


Figura 10. Validacion correcta del vehiculo por placa e idUsuario.

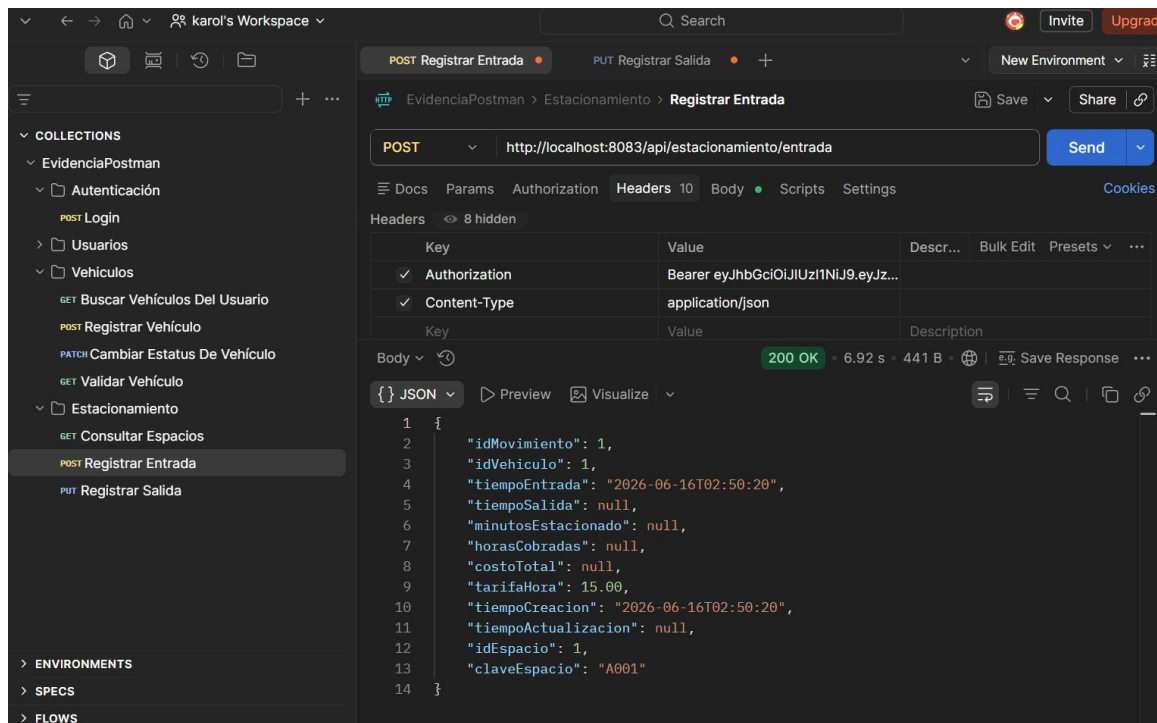


Figura 11. Registro de entrada con idMovimiento y espacio asignado.

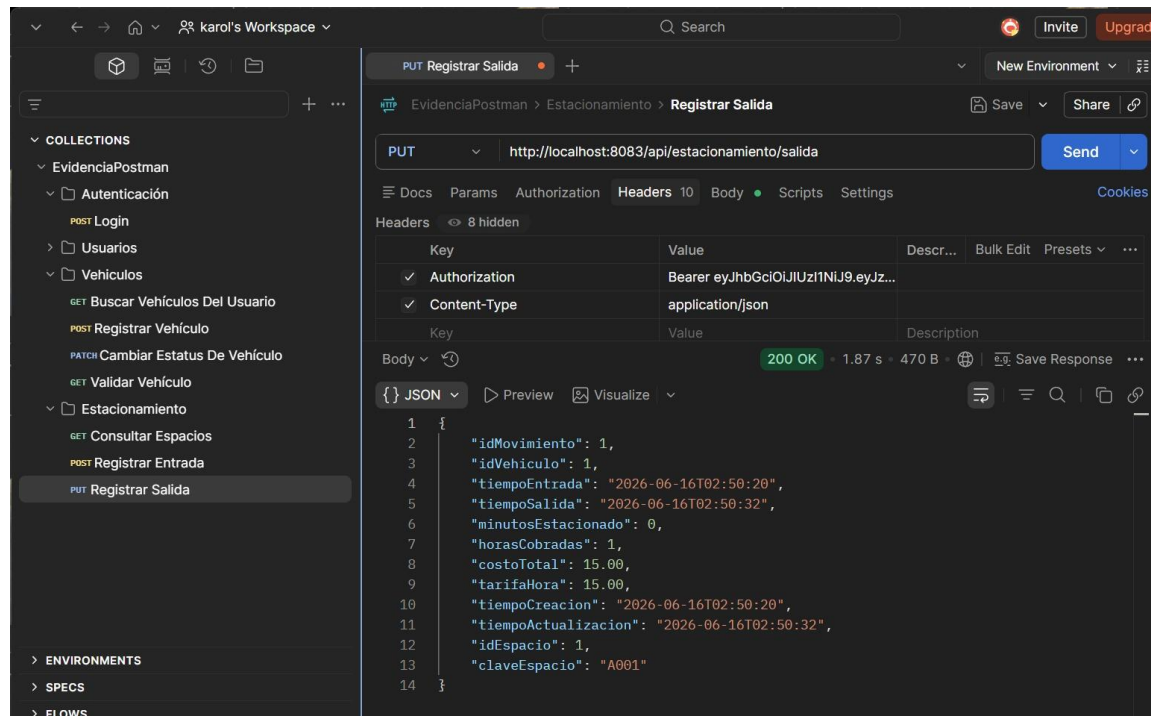


Figura 12. Registro de salida con calculo de costo total.

## Casos de error

También se revisaron escenarios de error para comprobar que el sistema respondiera cuando la información no era válida o cuando faltaba alguna condición necesaria. Estas pruebas son importantes porque muestran que el sistema no solo funciona en el camino ideal, sino que también detecta situaciones incorrectas.

Entre los errores revisados se encuentran credenciales incorrectas, problemas de conexión con base de datos durante la configuración inicial, peticiones protegidas con token incorrecto o ausente, y consultas a servicios antes de tener los datos necesarios registrados.

Estos errores ayudaron a ajustar la configuración del entorno, revisar dependencias, validar el orden correcto de pruebas y confirmar que las peticiones deben ejecutarse con el token en el encabezado Authorization.

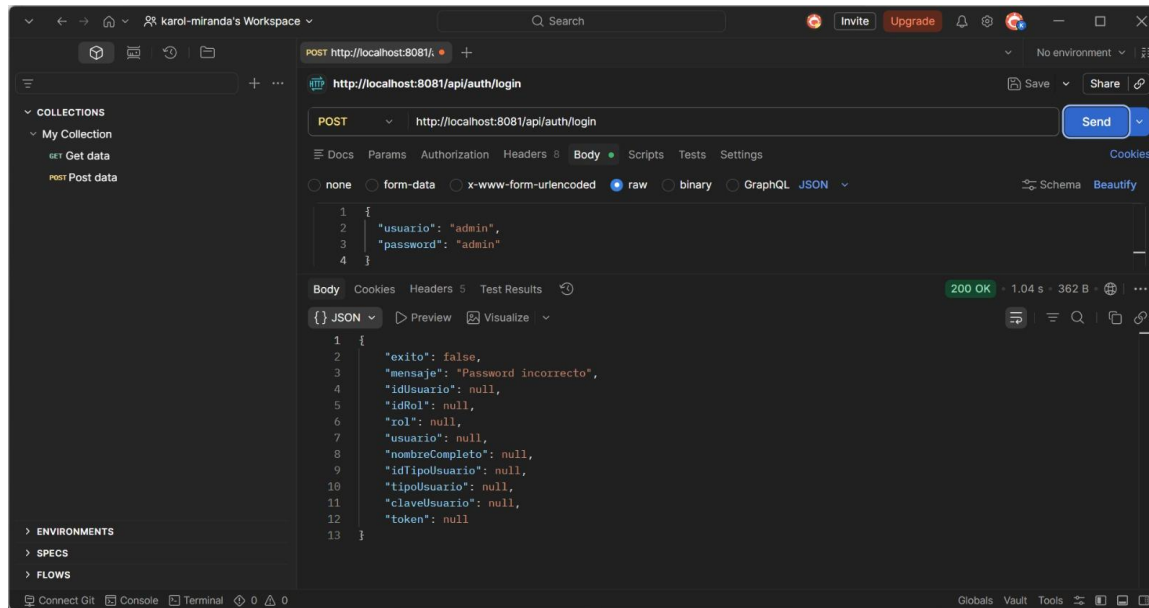


Figura 13. Caso de error controlado: login con password incorrecto.

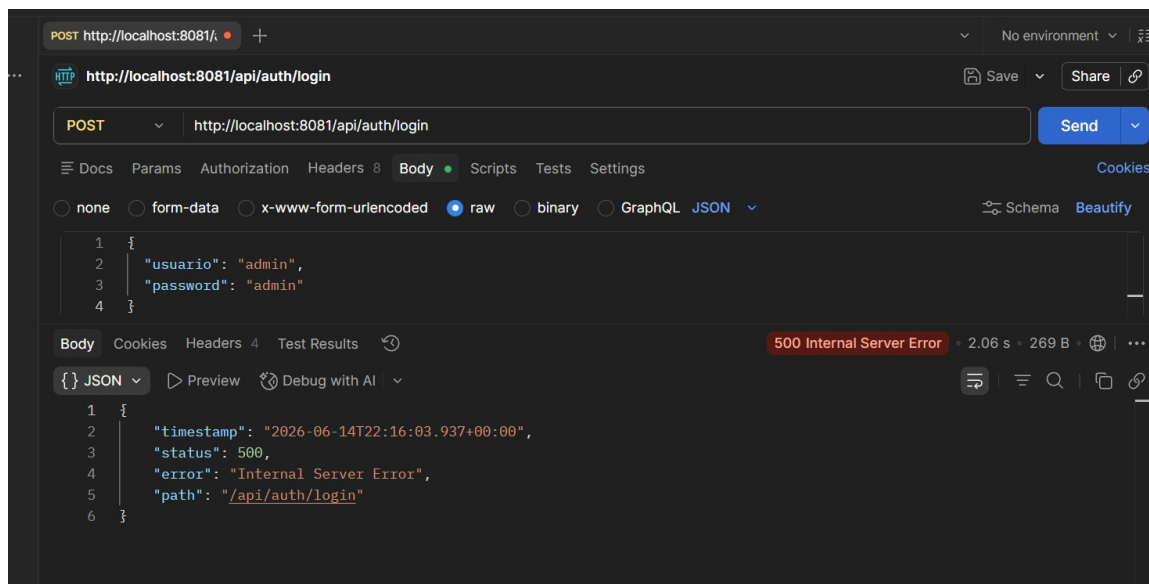


Figura 14. Error inicial de servidor durante pruebas de conexión y configuración.



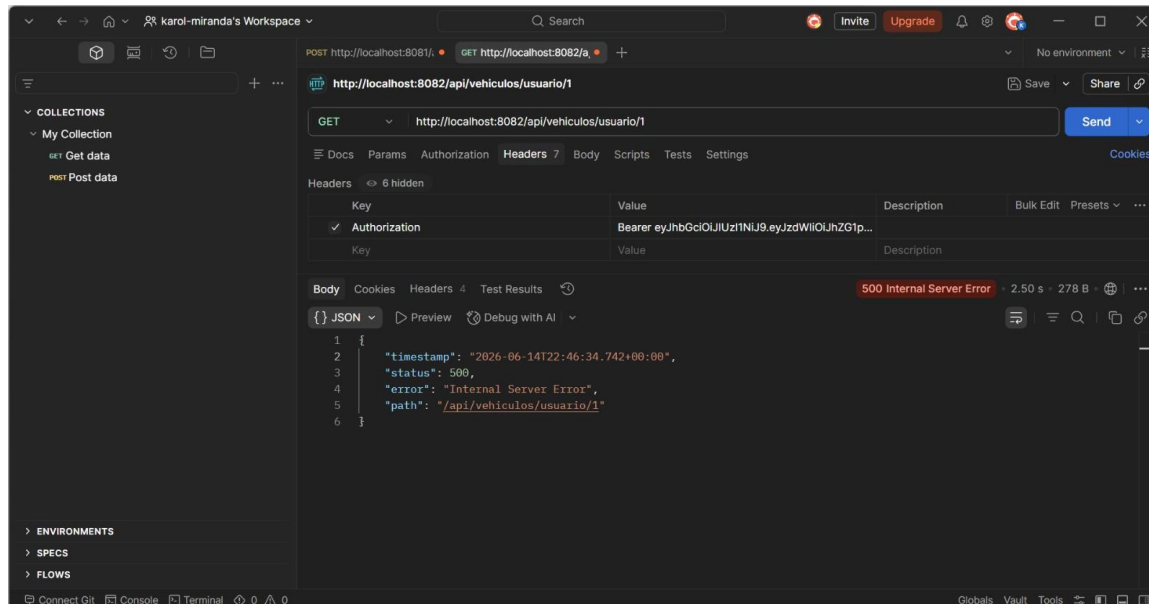


Figura 15. Error al consultar vehiculos antes de corregir la conexion del servicio.

## Evidencias

En este apartado insertamos las capturas de Postman y Docker obtenidas durante las pruebas, incluyendo: login exitoso, registro de vehículo, validación de vehículo, registro de entrada, registro de salida, lista de contenedores ejecutándose y logs donde se observe que los microservicios iniciaron correctamente.

```
[+] Running 3/3
✔projectosicac-sicac-estacionamiento Built 0.0s
✔Network projectosicac-sicac-servidor3 Created 0.3s
[+] Running 5/5
✔projectosicac-sicac-estacionamiento_data Created 0.1s
✔projectosicac-sicac-estacionamiento Built 0.0s
✔Network projectosicac-sicac-servidor3 Created 0.3s
✔Volume "projectosicac-sicac-estacionamiento_data" Created 0.1s
✔Container sicac-db-estacionamiento Healthy 23.2s
✔Container sicac-estacionamiento Started 23.9s
PS C:\Users\karol\Downloads\ProyectoMicroservicios\ProyectoSICAC> docker ps
```

| CONTAINER ID                    | IMAGE                               | NAMES                               | COMMAND                  | CREATED        | STATUS                  | PORTS        |
|---------------------------------|-------------------------------------|-------------------------------------|--------------------------|----------------|-------------------------|--------------|
| fb5762c6ccd4                    | projectosicac-sicac-estacionamiento | projectosicac-sicac-estacionamiento | "java -jar app.jar"      | 30 seconds ago | Up 6 seconds            | 0.0.0.0:8083 |
| ->8083/tcp, [::]:8083->8083/tcp | sicac-estacionamiento               |                                     |                          |                |                         |              |
| a49b16935ae5                    | mysql:8.0                           | sicac-db-estacionamiento            | "docker-entrypoint.s..." | 30 seconds ago | Up 28 seconds (healthy) | 0.0.0.0:3308 |
| ->3306/tcp, [::]:3308->3306/tcp | sicac-db-estacionamiento            |                                     |                          |                |                         |              |
| 77a8246ee52d                    | projectosicac-sicac-vehiculos       | projectosicac-sicac-vehiculos       | "java -jar app.jar"      | 3 minutes ago  | Up 2 minutes            | 0.0.0.0:8082 |
| ->8082/tcp, [::]:8082->8082/tcp | sicac-vehiculos                     |                                     |                          |                |                         |              |
| 7acbd32f3817                    | mysql:8.0                           | sicac-db-vehiculos                  | "docker-entrypoint.s..." | 3 minutes ago  | Up 3 minutes (healthy)  | 0.0.0.0:3307 |
| ->3306/tcp, [::]:3307->3306/tcp | sicac-db-vehiculos                  |                                     |                          |                |                         |              |
| d967a5690f0e                    | projectosicac-sicac-usuarios        | projectosicac-sicac-usuarios        | "java -jar app.jar"      | 5 minutes ago  | Up 4 minutes            | 0.0.0.0:8081 |
| ->8081/tcp, [::]:8081->8081/tcp | sicac-usuarios                      |                                     |                          |                |                         |              |
| 71660b813a69                    | postgres:17                         | sicac-db-usuarios                   | "docker-entrypoint.s..." | 5 minutes ago  | Up 5 minutes (healthy)  | 0.0.0.0:5433 |
| ->5432/tcp, [::]:5433->5432/tcp | sicac-db-usuarios                   |                                     |                          |                |                         |              |

```
PS C:\Users\karol\Downloads\ProyectoMicroservicios\ProyectoSICAC>
```

Figura 16. Contenedores de usuarios, vehiculos y estacionamiento ejecutandose en Docker.



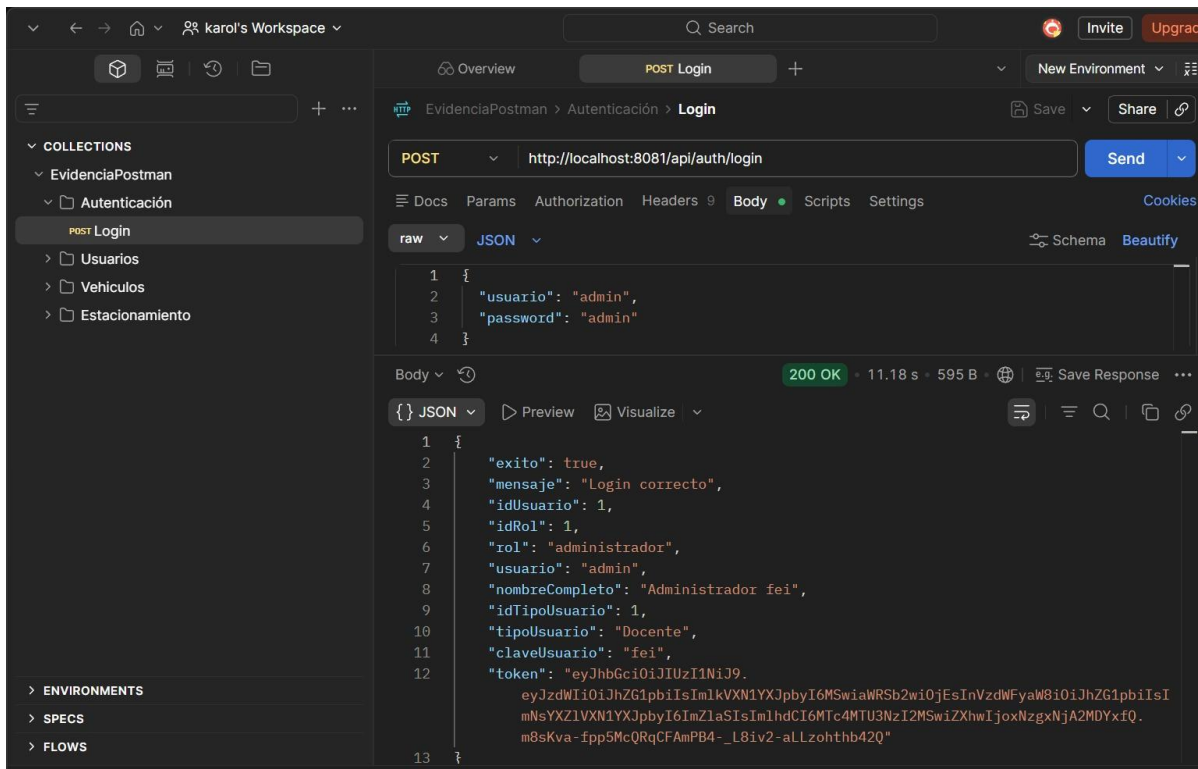


Figura 17. Login exitoso en Postman y generacion del token JWT.

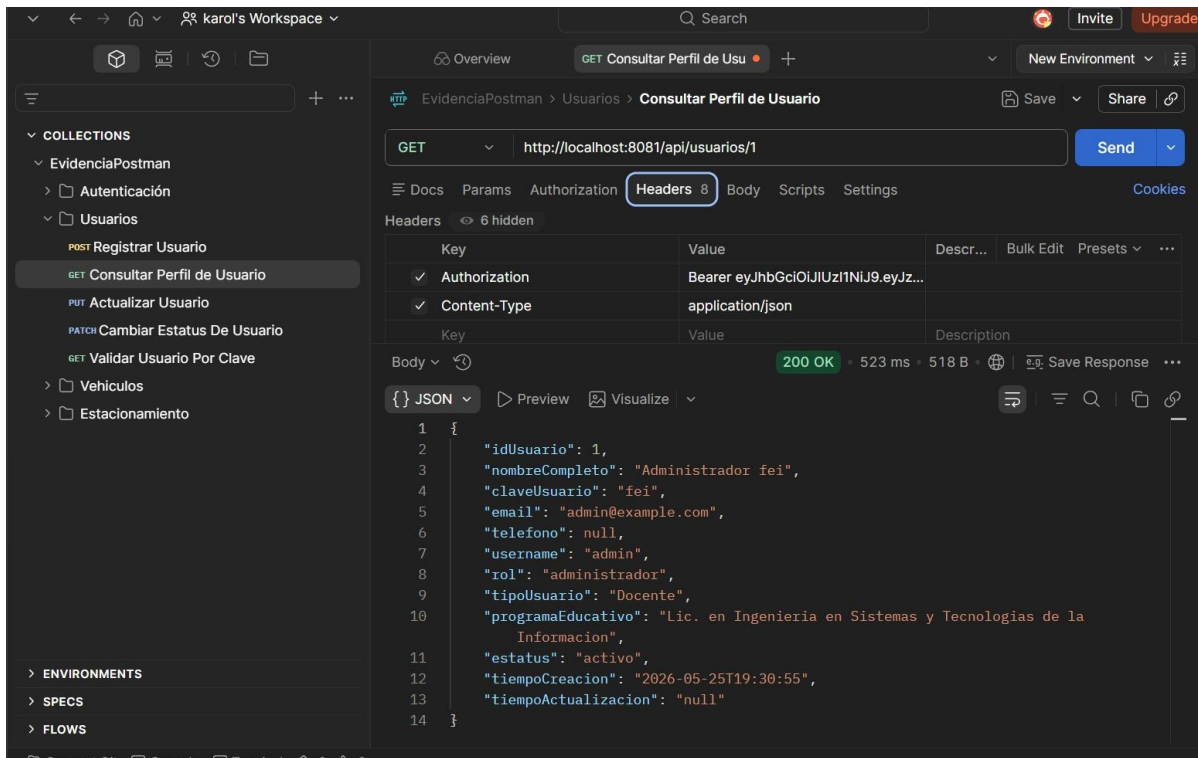


Figura 18. Consulta del perfil de usuario utilizando el encabezado Authorization.

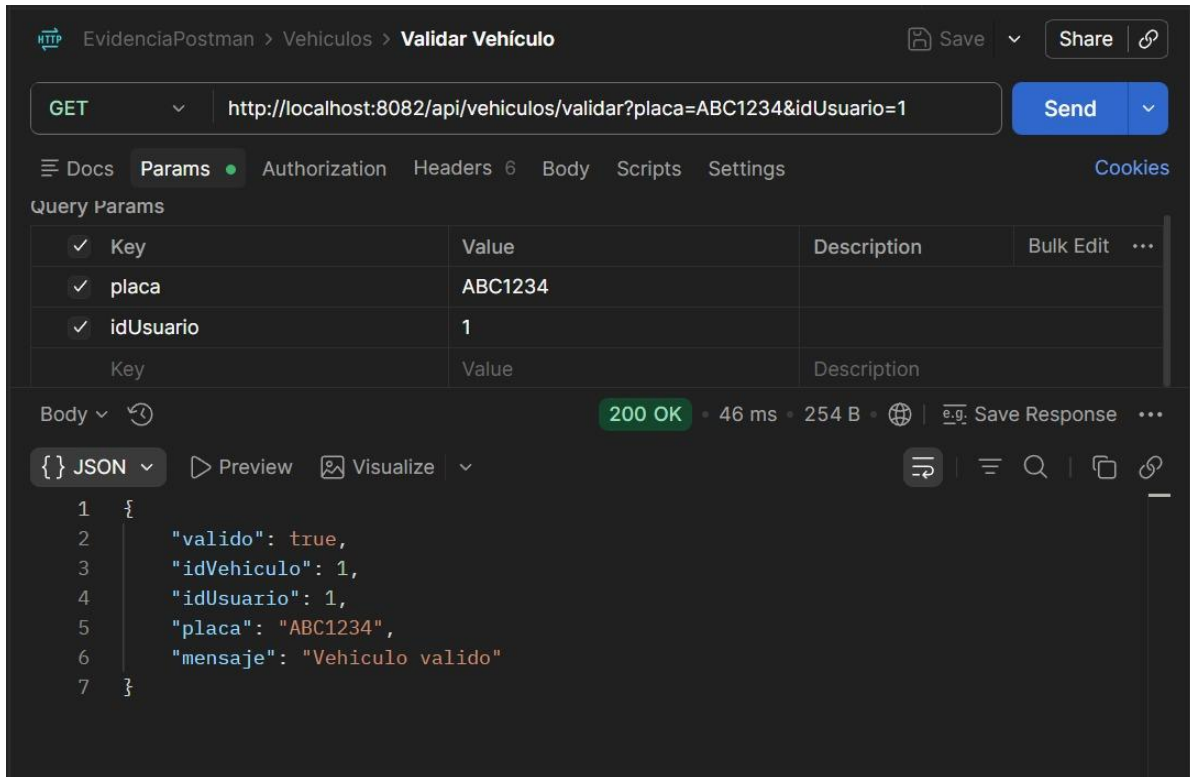


Figura 19. Validacion correcta del vehiculo por placa e idUsuario.

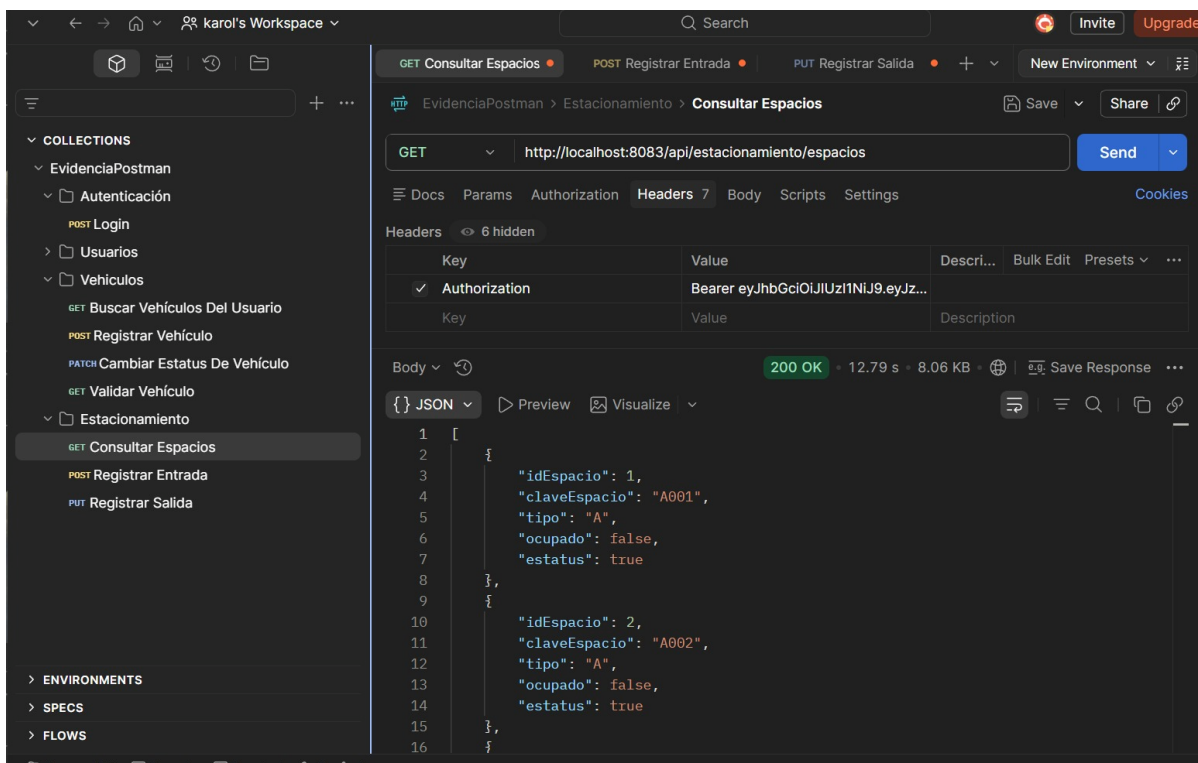


Figura 20. Consulta de espacios de estacionamiento disponibles.



*Universidad Veracruzana*  
*Facultad de Estadística e Informática*  
*Licenciatura en Ingeniería en Sistemas y Tecnologías de la Información*  
*Experiencia Educativa Tecnologías para la Integración de Soluciones*  
*Documentación Proyecto Final*  
*Periodo Febrero – Julio 2026*

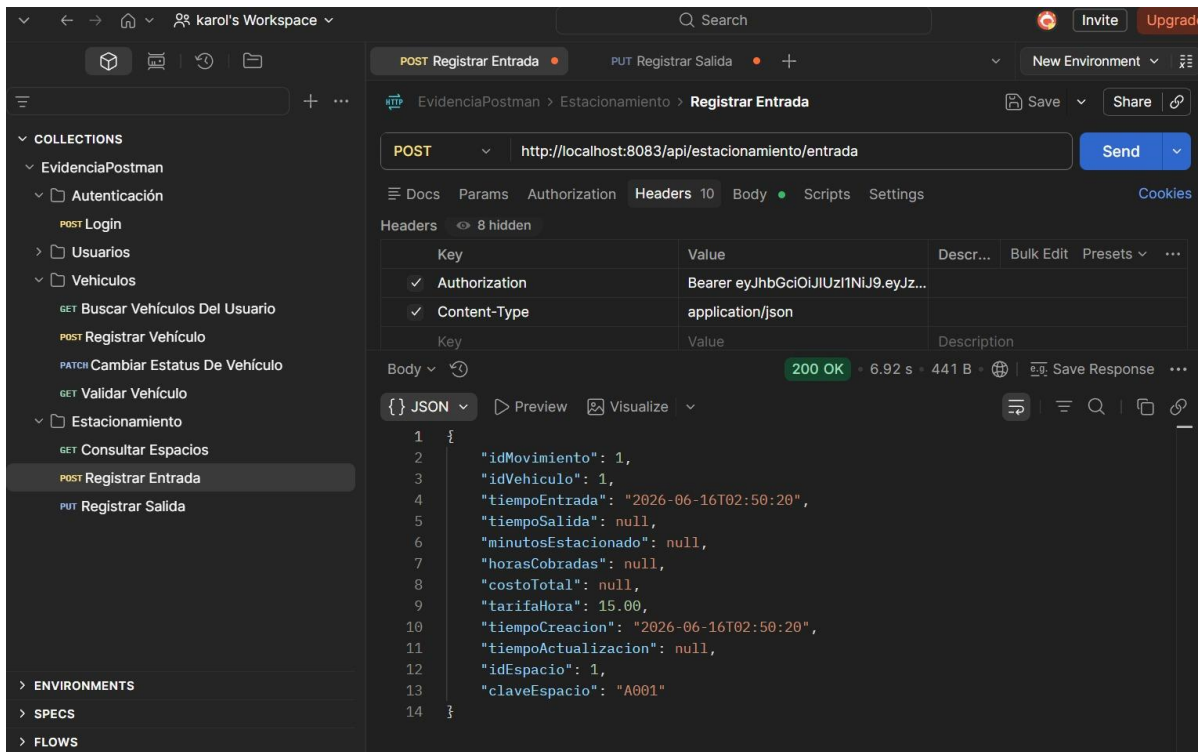


Figura 21. Registro exitoso de entrada al estacionamiento.

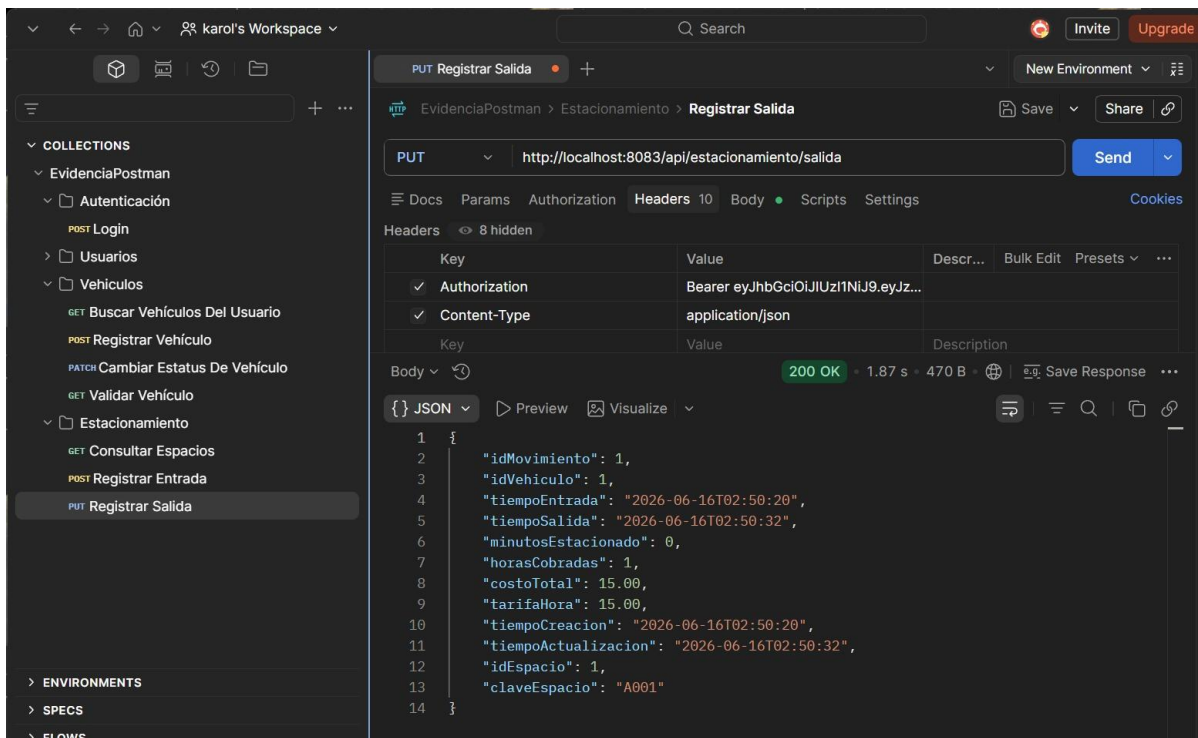


Figura 22. Registro exitoso de salida y calculo del costo total.



## Conclusiones

### Resultados obtenidos

Se obtuvo un sistema funcional dividido en tres microservicios. Los servicios pueden ejecutarse desde NetBeans para pruebas locales y también mediante Docker Compose para el despliegue por servidores.

El flujo completo fue probado: autenticación, registro y validación de vehículo, entrada al estacionamiento y salida con cálculo de costo.

### Aprendizajes

Durante el desarrollo se reforzó el uso de servicios REST, conexión a bases de datos, separación por capas, uso de JWT, manejo de scripts SQL y despliegue con Docker Compose.

También se comprendió la diferencia entre ejecutar un proyecto localmente en NetBeans y desplegarlo en contenedores, especialmente en el manejo de puertos, nombres de servicios, variables de entorno y volúmenes.

### Problemas encontrados

Durante el desarrollo del proyecto se presentaron varios problemas propios de trabajar con tres proyectos separados en NetBeans. Al inicio fue necesario revisar con cuidado los nombres de paquetes, clases principales y rutas de los archivos, porque un error pequeño en el package o en el nombre del proyecto provocaba que NetBeans marcara paquetes inexistentes o que Spring Boot no detectara correctamente algunos componentes.

Otro problema fue la configuración de dependencias en Maven. En algunos momentos el proyecto no reconocía anotaciones o librerías necesarias, por lo que se tuvo que revisar el archivo pom.xml, actualizar dependencias y ejecutar Clean and Build para confirmar que cada microservicio compilara de forma independiente antes de probarlo con Postman.

También hubo dificultades al decidir qué tecnologías usar pues no habíamos hecho un proyecto así de completo en clase, por lo que tomamos inspiración de actividades y de internet. Se revisaron opciones para la conexión a base de datos y finalmente se mantuvo MyBatis porque permite escribir consultas SQL de forma directa, lo que facilita explicar qué hace cada repositorio y cómo se relaciona con las tablas.

Durante las pruebas de APIs también aparecieron errores comunes, como respuestas 400 o 500 cuando faltaba el encabezado Authorization, cuando el token no se enviaba con el formato Bearer, o cuando se probaba un endpoint antes de registrar los datos necesarios. Esto ayudó a ordenar mejor la secuencia de pruebas: login, copia del token, registro de vehículo, validación, entrada y salida.

Para el despliegue con Docker Compose, uno de los retos fue redactar, o más bien adaptar la sintaxis YAML de los ejemplos en línea a nuestro proyecto correctamente. Los espacios, rutas relativas, nombres de servicios, puertos y variables de entorno tenían que estar bien definidos, ya que un error en cualquiera de esos puntos impedía levantar los contenedores o conectar el microservicio con su base de datos.